Name: Samir Kumar Gupta Roll No.: 2301730243 Environment: Ubuntu 24.04 LTS (OrbStack VM, arm64)

Question 7 - Monitoring, logging and DevSecOps report

Question 7: Prepare a monitoring, logging, and DevSecOps report describing the automation, observability setup, security findings, and observations with screenshots.

All commands below were run on the lab machine and their output is copied verbatim from the recorded logs in `evidence/logs/`. Repository: https://github.com/ksamirdev/ensp461-devops-lab3.

1. Objective

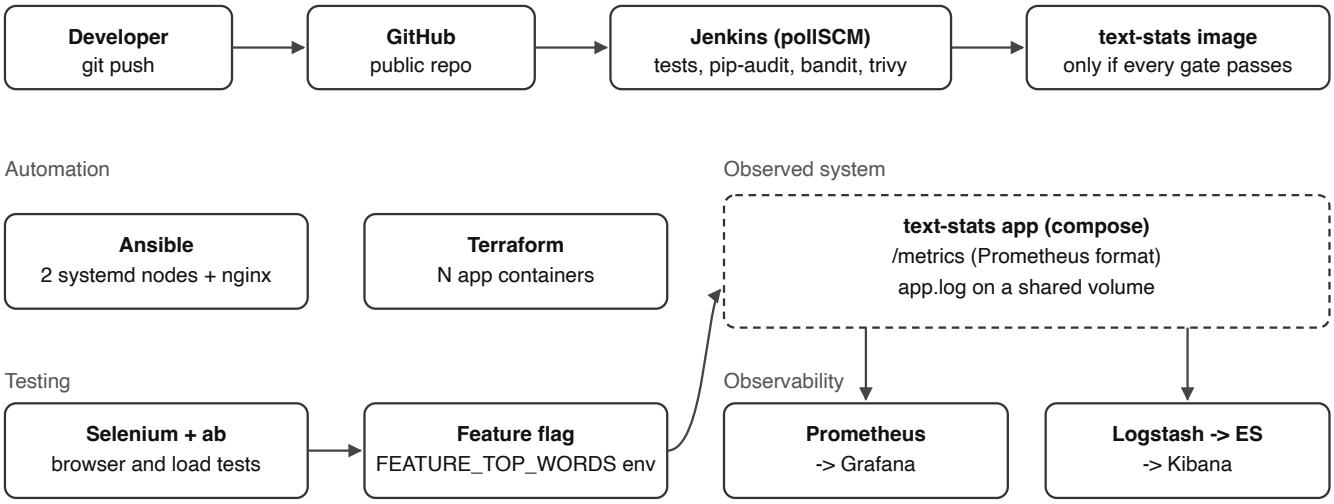
Take the text statistics service from Lab 1 and build an automated, observable and secure workflow around it. Ansible and Terraform handle automation, Prometheus and Grafana collect metrics, and the ELK stack centralizes logs. A Jenkins pipeline blocks insecure changes, and feature flags, load tests and browser tests check behaviour.

2. Environment and tools

```
$ cat /etc/os-release | head -1; uname -srm
PRETTY_NAME="Ubuntu 24.04.5 LTS"
Linux 7.2.12-orbstack-00394-gbecdd6130068 aarch64
$ docker version --format 'Docker Engine {{.Server.Version}}'; docker compose version
Docker Engine 29.9.3
Docker Compose version v5.7.0
$ ansible --version | head -1; terraform version | head -1; trivy --version | head -1
ansible [core 2.16.3]
Terraform v1.17.1
Version: 0.76.2
$ ~/lab-tools/bin/pip-audit --version; ~/lab-tools/bin/bandit --version | head -1; ab -V | head -1
pip-audit 2.10.1
bandit 1.9.4
This is ApacheBench, Version 2.3 <$Revision: 1903618 $>
$ docker exec lab3-pipeline-jenkins-1 cat /var/jenkins_home/war/META-INF/MANIFEST.MF | grep Jenkins-Version
Jenkins-Version: 2.580.2
$ docker images --format '{{.Repository}}:{{.Tag}}' | grep -E 'prometheus|grafana|elastic|selenium|jenkins' | sort
docker.elastic.co/elasticsearch/elasticsearch:9.5.6
lab3-pipeline-jenkins:latest
mirror.gcr.io/grafana/grafana:13.3.0
mirror.gcr.io/jenkins/jenkins:lts-jdk21
mirror.gcr.io/selenium/standalone-chromium:latest
quay.io/prometheus/prometheus:v3.16.0
```

Monitoring and logging images: Prometheus v3.16.0, Grafana 13.3.0, and Elasticsearch, Logstash and Kibana 9.5.6. Every port binds to 127.0.0.1 on the lab machine.

3. Architecture

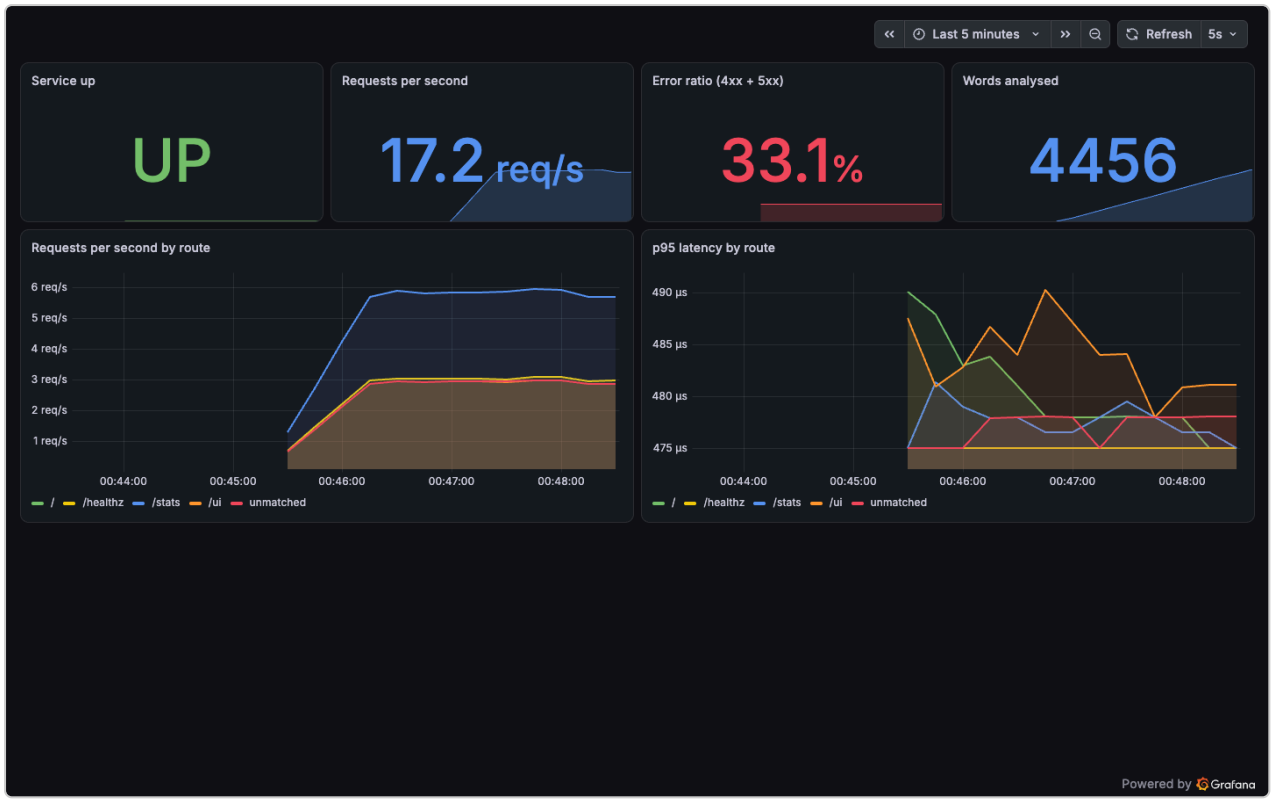


4. Automation

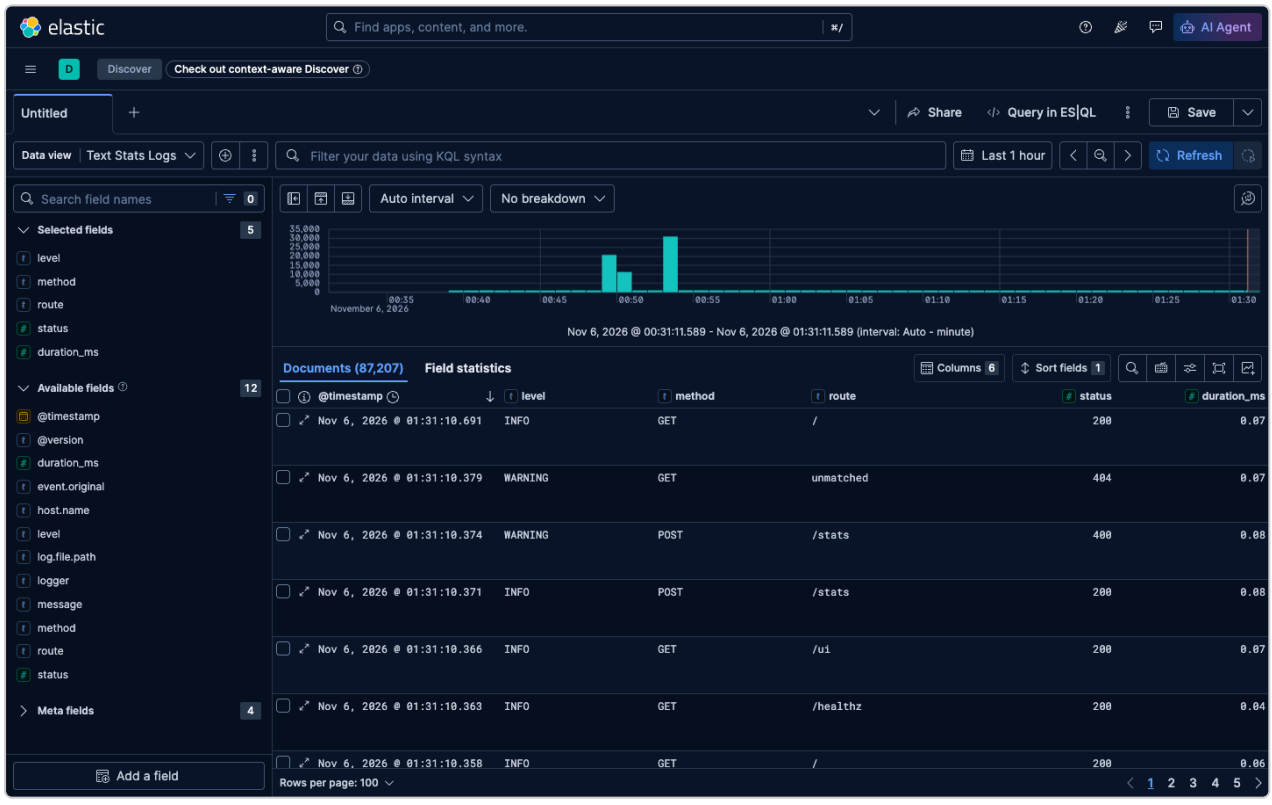
- **Ansible (Q1):** one playbook takes two bare Ubuntu nodes to a running service behind nginx. First run: changed=10 per node. Second run: changed=0.
- **Terraform (Q2):** container count is one variable. Scaling 2 to 4 added 2 containers and destroyed 0. A final `plan -detailed-exitcode` returned 0 (no drift).
- **Pipeline (Q5):** Jenkins polls GitHub and runs every gate on each push, with no manual trigger.

5. Observability

- **Metrics (Q3):** request counter by method, route and status, a latency histogram, and words processed. Prometheus scrapes every 5 s, and Grafana is provisioned from files with 6 panels.
- **Logs (Q4):** one structured line per request, shipped by Logstash, parsed with grok into level, method, route, status and duration fields, and explored in Kibana.
- Metrics and logs answer different questions. Grafana shows *how much* and *how fast*. Kibana shows *which request*: for example, every 400 for a non-string `text`.



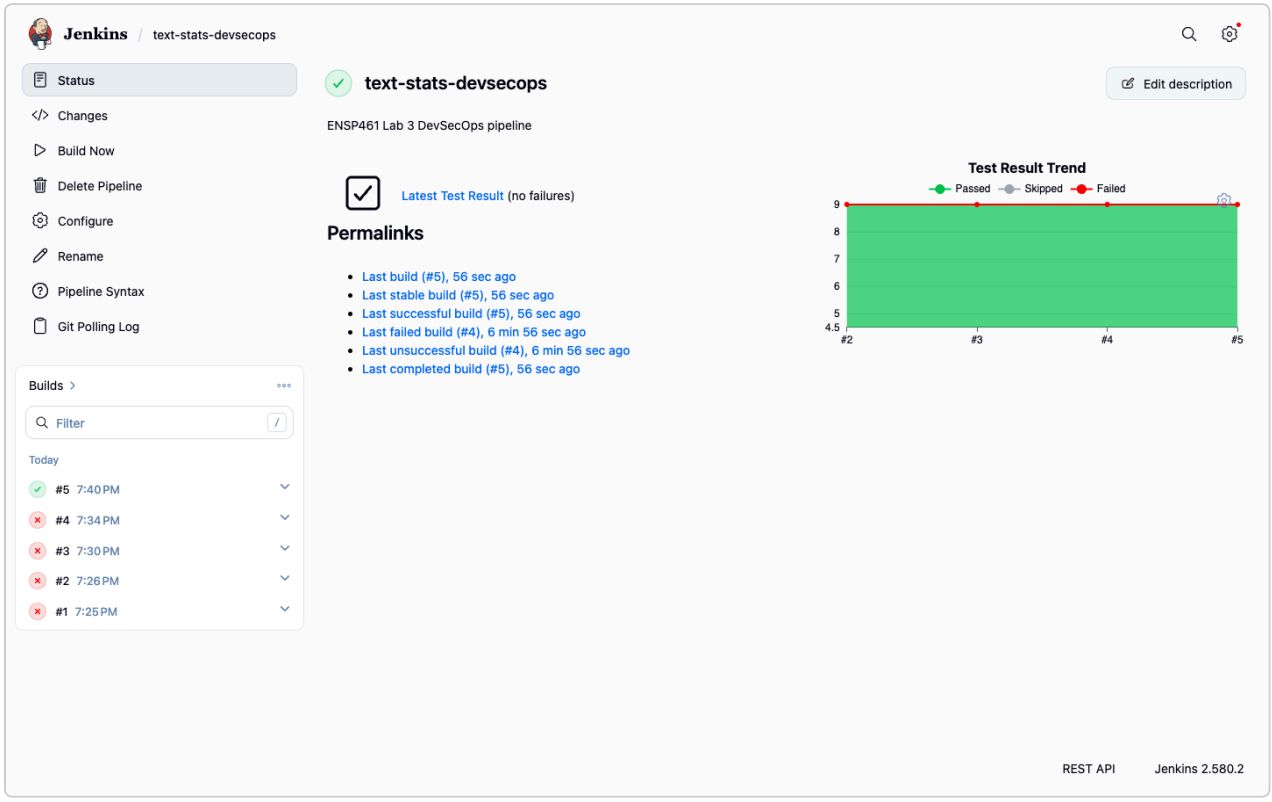
Grafana dashboard under traffic (Q3)



Kibana Discover over the parsed application logs (Q4)

6. Security findings

Gate	Finding	Action
pip-audit (build 2)	flask 3.1.0, pytest 8.3.4, urllib3 2.4.0: 20 advisories	Upgraded to patched versions
Trivy misconfig (build 3)	DS-0029 x2, DS-0002 x2 in the lab infrastructure Dockerfiles	Fixed DS-0029; DS-0002 accepted with reasons, scoped to the 2 files
Trivy image (build 4)	5 HIGH CVEs in pip's vendored msgpack, urllib3 and setuptools	Removed pip from the runtime image
Bandit (all builds)	No issues in `service/`	-
Trivy secrets (all builds)	No secrets in the public repository	-



Jenkins: builds 2-4 blocked by real findings, build 5 passed

7. Testing

- 9 unit tests, including metric label cardinality and both states of the feature flag.
- The feature flag toggles `/stats/top` between HTTP 404 and 200 without a rebuild.
- ApacheBench: about 4,000 req/s at concurrency 50 with 0 failures. At concurrency 200, p99 latency rose to 110 ms, while server-side p95 stayed under 0.5 ms (time spent queueing).
- Selenium found a bug the unit tests could not: a `const top` collision with `window.top` stopped the page script from loading.

8. Observations

- Every gate caught something real. Dependencies copied from an earlier lab had picked up advisories, and the biggest image finding came from a tool (pip) the app never uses.

- Defaults hide things: Docker's injected log options made Terraform replace containers, and Prometheus' default buckets flattened p95 latency. Both looked fine until they were checked against real data.
- Structured log lines are what make ELK useful. Grok turns each line into typed fields, so Kibana can count and filter instead of doing full-text search.
- Security settings are deliberately relaxed for the lab (no auth on Elasticsearch and Jenkins, anonymous Grafana viewer). Every port binds to localhost to keep that safe. Production would need auth and TLS.